

# Analyse de Sentiment sur Textes Financiers

---

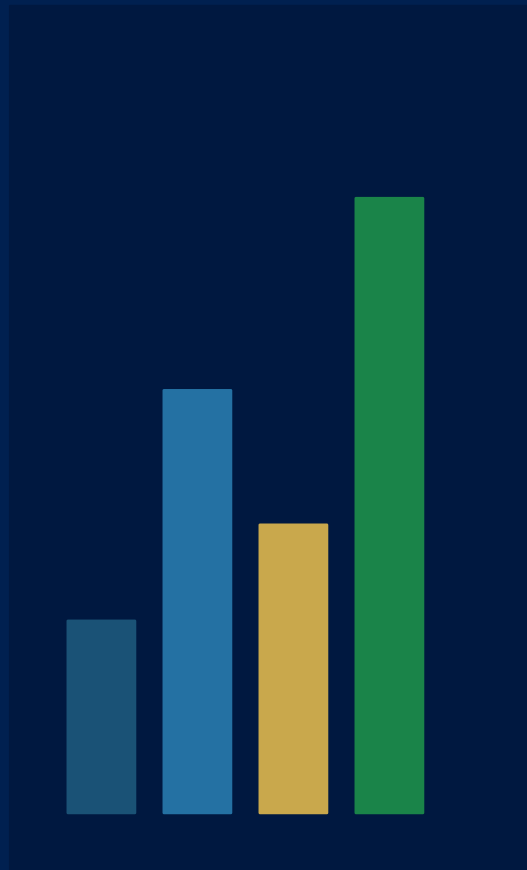
*Projet Deep Learning & NLP*

*Benchmarking Classical ML, Deep Learning et Transformers*

**Housseem Majed · Victoria Vidal**

Master MoSEF — Deep Learning / NLP

Université Paris 1 Panthéon-Sorbonne — Avril 2026



# Plan de la présentation

01

## Contexte & Objectifs

Problématique et enjeux de l'analyse de sentiment financier

02

## Dataset & Protocole

FinancialPhraseBank · Split stratifié · Métriques

03

## Modèles développés

Classical ML · BiLSTM · Transformers (BERT, DistilBERT, FinBERT)

04

## Résultats & Analyse

Benchmark comparatif · Enseignements clés

05

## Dashboard Streamlit

Fonctionnalités · LIME · Live News · Entrée vocale

06

## Difficultés & Conclusion

Challenges techniques · Perspectives

07

## Démonstration

Dashboard en conditions réelles

## Problématique

Les marchés financiers produisent chaque jour un volume massif de textes (communiqués, rapports d'analystes, dépêches Reuters). Classifier automatiquement leur sentiment permet d'extraire un signal utile pour l'aide à la décision.

Objectif : comparer 3 générations d'approches NLP sur une tâche de classification de sentiment financier en 3 classes

## 1. Classical ML

TF-IDF + classifieurs linéaires  
(LogReg, Naive Bayes)

## 2. Deep Learning

Bidirectional LSTM  
(embeddings appris)

## 3. Transformers

BERT · DistilBERT · FinBERT  
(pré-entraînés, fine-tunés)

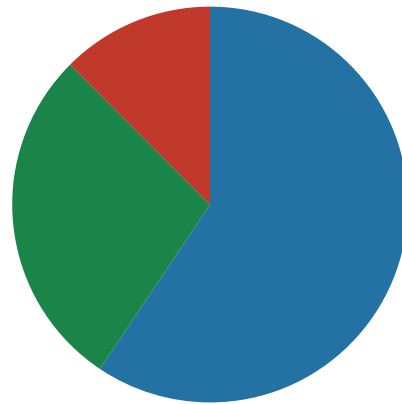
**Source :** Malo et al. (2014), JASIST 65(4)  
**4 846 phrases** extraites de communiqués financiers (entreprises européennes),  
annotées par **16 experts en finance**.

**Question d'annotation :** « Cette phrase donne-t-elle une impression positive, négative ou neutre de l'entreprise mentionnée ? »

Seules les phrases avec accord majoritaire (>50%) sont retenues.

**Split stratifié (seed=42) :**

| Split      | Échantillons | Part |
|------------|--------------|------|
| Train      | 3 391        | 70%  |
| Validation | 485          | 10%  |
| Test       | 970          | 20%  |



■ Neutral (59.4%) ■ Positive (28.1%) ■ Negative (12.5%)

⚠ Déséquilibre important : la classe négative ne représente que 12.5% du dataset → impacte directement le choix de la métrique d'évaluation.

## Pourquoi le Macro-F1 ?

Un modèle qui prédit toujours « neutral » obtient 59% d'accuracy — mais un macro-F1 proche de 0. La classe négative (12.5%) pèse autant que la classe neutre (59.4%) dans le calcul : c'est ce qui rend cette métrique indispensable sur des données déséquilibrées.

$$\text{Macro-F1} = (1/C) \times \sum F1c \quad \text{où} \quad F1c = 2 \cdot \text{Précision}_c \cdot \text{Rappel}_c / (\text{Précision}_c + \text{Rappel}_c)$$



**Équitable entre classes**

Chacune des 3 classes compte pour 1/3



**Accuracy = trompeuse**

BiLSTM : 63.7% acc. mais 37.3% macro-F1



**Recall sur la minorité**

0% recall sur «negative» → macro-F1 effondré

## 1. Classical ML

- TF-IDF (unigrammes + bigrammes, 20K features, TF sublinéaire)
- Logistic Regression — régularisation L2, solver lbfgs
- Naive Bayes — MultinomialNB,  $\alpha=0.1$
- Avantage : <1s d'entraînement, interprétable
- Limite : ignore l'ordre & le contexte

## 2. Deep Learning

- Embedding 64d (15K vocab) → BiLSTM 2 couches (128 unités) → Dense → Softmax
- 20 epochs, batch=32, Adam, early stopping
- ~40s sur GPU
- Avantage : capture l'ordre séquentiel
- Limite : 0% recall sur «negative» sans pré-entraînement — GPU ne compense pas le manque de données

## 3. Transformers

- Principe : pré-entraînement massif + fine-tuning
- BERT base-uncased (110M params) — BookCorpus + Wikipedia
- DistilBERT (66M params) — 40% plus léger
- FinBERT (110M) — pré-entraîné sur corpus financier
- Fine-tuning : 5 epochs,  $lr=2e-5$ , early stopping

GPU : NVIDIA RTX 5070 Laptop (8 Go VRAM) — optimisations mémoire requises pour le fine-tuning des Transformers

## Modules src/

### **config.py**

chemins, hyperparamètres, registre des modèles

### **data.py**

chargement, nettoyage, split stratifié

### **metrics.py**

accuracy, macro-F1, weighted-F1

### **inference.py**

chargement modèle, pipeline HuggingFace, prédiction

## Scripts d'entraînement

### **train\_baselines.py**

TF-IDF + LogReg / Naive Bayes

### **train\_lstm.py**

BiLSTM via TensorFlow / Keras

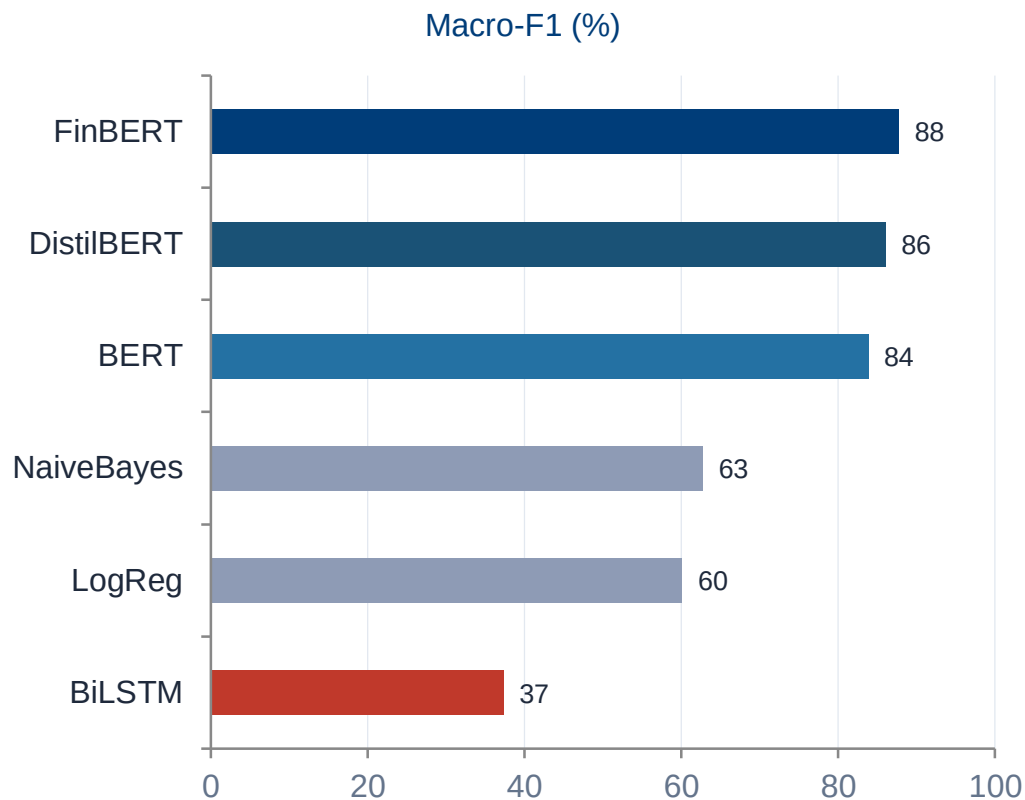
### **train\_transformer.py**

BERT / DistilBERT / FinBERT (HuggingFace)

### **build\_leaderboard.py**

Agrégation résultats → JSON / CSV / Markdown

# Résultats — Benchmark comparatif (970 exemples test)



| # | Modèle     | Acc.  | Macro-F1 | Temps |
|---|------------|-------|----------|-------|
| 1 | FinBERT    | 88.1% | 87.7%    | 82s   |
| 2 | DistilBERT | 86.8% | 86.0%    | 51s   |
| 3 | BERT       | 85.5% | 83.8%    | 111s  |
| 4 | NaiveBayes | 71.3% | 62.7%    | <1s   |
| 5 | LogReg     | 72.4% | 60.1%    | <1s   |
| 6 | BiLSTM     | 63.7% | 37.3%    | 41s   |

FinBERT domine grâce au pré-entraînement financier (+3.9pts vs BERT)

DistilBERT > BERT : modèle plus compact = moins d'overfitting

Baselines : accuracy correcte mais échec sur la classe minoritaire

BiLSTM : 0% recall sur «negative» sans pré-entraînement



*Interface web complète avec 8 pages (Analyze · Batch Analysis · Live News · Leaderboard · Dataset · Methodology · Rendu · About) — design CSS personnalisé, modèles mis en cache via `st.cache_resource`.*

## Analyze

- Saisie textuelle ou vocale (Web Speech API)
- Prédiction avec modèle au choix
- Comparaison 3 transformers côte à côte
- Explicabilité LIME (surlignage + barres)

## Batch Analysis

- Upload CSV de headlines
- Classification en masse
- Graphiques : camembert + histogramme confiance
- Export des résultats en CSV

## Live News

- Saisie d'un ticker boursier (AAPL, TSLA...)
- API yfinance → dernières headlines
- Classification en temps réel
- Score de sentiment global (positifs – négatifs)

## Leaderboard

- `build_leaderboard.py` → JSON/CSV/Markdown
- Tableau trié par macro-F1 décroissant
- Accuracy · Macro-F1 · Weighted-F1 · Temps
- Mis à jour automatiquement

## Explicabilité avec LIME

**LIME (Ribeiro et al., 2016)** génère des perturbations locales de la phrase (en masquant des mots) et observe l'évolution de la prédiction. Les mots les plus influents sont identifiés et affichés de deux façons :

- ① **Surlignage coloré** (vert = en faveur de la classe prédite, rouge = contre)
- ② **Diagramme à barres horizontales** avec score d'importance quantitatif pour chaque mot.

## Entrée vocale dans Streamlit — Défi technique

**Problème** : les iframes Streamlit (sandbox) bloquent l'accès au microphone.

**Solution** : utiliser `components.html()` pour injecter un `<script>` dans le document parent Streamlit → le SpeechRecognition s'exécute dans le bon contexte avec les permissions navigateur.

**Synchronisation** : setter natif de `HTMLTextAreaElement` + événements synthétiques `input/change` + callbacks `on_click` pour que Streamlit détecte la nouvelle valeur.

1

## Gestion VRAM GPU

Fine-tuning BERT/FinBERT (110M params) sur RTX 5070 8Go → fp16 + gradient checkpointing + gradient accumulation + cap 85%. Sans ces optimisations : CUDA out-of-memory.

2

## PyTorch CUDA sur Windows

pip install torch donne une version CPU-only. Solution : index pytorch-cu128 configuré dans pyproject.toml via [tool.uv.sources].

3

## Entrée vocale dans Streamlit

Les iframes sandboxées bloquent le micro. Plusieurs approches échouées (st\_javascript, components.html direct) avant de trouver l'injection dans le contexte parent.

4

## Synchronisation JS ↔ Streamlit

Modification textarea via JS non détectée par Streamlit → setter natif HTMLTextAreaElement + événements synthétiques + callbacks on\_click.

5

## Déséquilibre des classes

BiLSTM prédit systématiquement «neutral» → 0% recall sur «negative». Confirmation que le pré-entraînement est indispensable sur des datasets spécialisés de petite taille.

# Conclusion & Perspectives

**FinBERT domine le benchmark** avec un macro-F1 de **87.7%**. Le pré-entraînement spécifique au domaine financier est le facteur déterminant sur un petit dataset spécialisé. LIME confirme au niveau des mots que FinBERT identifie les bons signaux sémantiques là où les modèles génériques peuvent se laisser égarer.

## Enseignements clés

- Le paradigme « pre-train then fine-tune » est décisif quand les données sont rares
- DistilBERT > BERT sur petit dataset : la compacité réduit l'overfitting
- L'accuracy seule est trompeuse → toujours reporter le macro-F1
- BiLSTM : architecture pertinente mais insuffisante sans pré-entraînement

## Perspectives

- Élargir le dataset (SEC filings, earnings calls)
- DeBERTa, LLMs few-shot (GPT-4, Mistral)
- Déploiement cloud + API REST
- Analyse multi-langue (français, allemand)

Merci de votre attention — Place à la démonstration